

- 基于Spark ALS的大规模离线电影推荐系统
 - 实验报告
 - 一、绪论
 - 二、算法设计
 - 2.1 ALS矩阵分解算法
 - 2.1.1 矩阵分解原理
 - 2.1.2 损失函数与优化目标
 - 2.1.3 交替优化过程
 - 2.1.4 Spark分布式并行实现
 - 2.2 冷启动内容推荐算法
 - 2.3 混合推荐算法
 - 三、实验设计
 - 3.1 数据集
 - 3.2 评价指标
 - 3.3 实验方案
 - 四、实验结果与分析
 - 4.1 ALS基线 (A)
 - 4.2 超参数搜索 (B)
 - 4.3 冷启动内容推荐 (C)
 - 4.4 混合推荐 (D)
 - 4.5 规模化验证 (E)
 - 五、综合分析
 - 算法改进与创新点
 - 方案对比分析
 - 六、总结与展望
 - 附录：核心算法代码及说明
 - 训练脚本 (train_als.py)
 - 服务脚本 (app.py)

基于Spark ALS的大规模离线电影推荐系统

实验报告

小组成员：23336156刘沛浩、23336168卢思源、23336136林华杰

一、绪论

随着互联网视频平台的快速发展，用户面对海量电影内容时往往难以做出选择。推荐系统作为信息过滤的关键技术，能够根据用户的历史行为自动发现其潜在兴趣。

本实验基于MovieLens数据集（1M和25M），使用Spark MLlib ALS构建离线电影推荐系统，针对冷启动问题引入内容推荐和混合推荐策略。系统采用离线训练+在线服务两层架构：离线层PySpark加载评分数据，进行ALS矩阵分解训练，支持超参数网格搜索、冷启动内容推荐和混合推荐；在线层Flask提供REST API，将CSV加载到内存或SQLite作为KV缓存。

核心技术栈：Python 3.11, PySpark 3.4.1, Spark MLlib ALS, Flask 2.3.0, Java 8 (Temurin 1.8.0_492)

二、算法设计

2.1 ALS矩阵分解算法

2.1.1 矩阵分解原理

ALS将用户-物品评分矩阵分解为两个低秩矩阵的乘积：

$$R_{m \times n} \approx U_{m \times k} \times V_{n \times k}^T$$

其中各矩阵的含义为：

- $R_{m \times n}$ ：用户-物品评分矩阵。 m 为用户数， n 为物品数。该矩阵极度稀疏（ml-1m稀疏度95.5%，ml-25m稀疏度99.8%）。
- $U_{m \times k}$ ：用户因子矩阵。每行 U_i 表示用户 i 在 k 个隐语义维度上的偏好强度。
- $V_{n \times k}$ ：物品因子矩阵。每行 V_j 表示物品 j 在 k 个隐语义维度上的属性特征。
- k ：隐因子维度（rank），本实验默认 $k = 50$ 。

预测评分通过内积计算：

$$\hat{r}_{ij} = U_i \cdot V_j^T = \sum_{f=1}^k u_{if} \cdot v_{jf}$$

2.1.2 损失函数与优化目标

$$\min_{U,V} \sum_{(i,j) \in \Omega} (R_{ij} - U_i V_j^T)^2 + \lambda \left(\sum_i \|U_i\|^2 + \sum_j \|V_j\|^2 \right)$$

第一项为拟合误差（仅对观测评分 Ω 计算），第二项为L2正则化， λ （regParam）控制正则化强度。

2.1.3 交替优化过程

ALS交替固定一个因子矩阵求解另一个，每一步是凸优化有闭式解。

Step 1 - 固定 V 求解 U ：

$$U_i = (V_{I_i}^T V_{I_i} + \lambda I_k)^{-1} V_{I_i}^T R_{i,I_i}$$

I_i 是用户 i 评分过的物品索引集合， I_k 是 $k \times k$ 单位矩阵。

Step 2 - 固定 U 求解 V ：

$$V_j = (U_{J_j}^T U_{J_j} + \lambda I_k)^{-1} U_{J_j}^T R_{J_j,j}$$

2.1.4 Spark分布式并行实现

1. **Block划分**：用户和物品按ID范围划分为多个Block，评分数据路由到交叉分区。
2. **按需拉取**：每个用户Block从BlockManager拉取所需 V Block到本地独立求解，无需全局Shuffle。
3. **缓存复用**：因子Block首次拉取后缓存到Executor本地，后续直接命中。

2.2 冷启动内容推荐算法

对于评分 <5 条的用户，采用Jaccard内容推荐：

$$\text{Jaccard}(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

用户口味画像 $G_u = \bigcup_{j \in R_u} \text{Genres}(j)$ 。对冷用户Cross Join计算Jaccard，left anti join排除已评分，取Top-N。

2.3 混合推荐算法

ALS生成 $top_n \times 5$ 候选池，计算Jaccard相似度，各评分min-max归一化：

$$\text{hybrid} = \alpha \cdot \text{norm}(\text{als}) + (1 - \alpha) \cdot \text{norm}(\text{content})$$

本实验 $\alpha = 0.7$ 。重排序取Top-N，冷用户走纯内容推荐。

三、实验设计

3.1 数据集

ml-1m: 6,040用户, 3,706电影, 1,000,209评分, 稀疏度95.5%, ::分隔符。 ml-25m: 162,541用户, 62,423电影, 25,000,095评分, 稀疏度99.8%, ,分隔符。

3.2 评价指标

RMSE (评分预测误差)、Precision@10 (精确率)、Recall@10 (召回率)、NDCG@10 (排序位置质量)、Coverage (覆盖率)、Diversity (列表内类型差异度)。相关电影定义为评分 ≥ 3.5 。

3.3 实验方案

- A (ALS基线): rank=50, regParam=0.1, maxIter=15, ml-1m
- B (网格搜索): rank {10,30,50} x regParam {0.01,0.1,0.5} x maxIter {10,15}, 3-fold CV
- C (冷启动): Jaccard类型相似度, ml-1m
- D (混合): alpha=0.7, ml-1m
- E (规模化): ml-25m, 最优参数, SQLite

四、实验结果与分析

4.1 ALS基线 (A)

RMSE=0.8697, Precision@10=0.3124, Recall@10=0.1758, NDCG@10=0.3382。冷启动丢弃2.3%，训练耗时184.5s。Precision@10=0.31说明Top-10中约3部是用户会高评价的。

4.2 超参数搜索 (B)

rank=50最优 (RMSE=0.870), rank=10不足 (0.912)。regParam=0.1最优。maxIter 10-15轮收敛。最优参数即默认配置。

4.3 冷启动内容推荐 (C)

ml-1m中8.8%用户为冷用户。内容推荐将冷用户覆盖率从0%提升至100%，整体Coverage从17.7%提升至28.5% (+10.8pp)。纯ALS仅推荐687部电影，这是ALS的"马太效应"——热门电影被反复推荐。

4.4 混合推荐 (D)

RMSE保持0.8697不变，Precision@10+5.2%，Recall@10+6.5%，NDCG@10+6.2%，Coverage+4.4pp，Diversity+9.4%。内容信号有效打破ALS热门偏向。

4.5 规模化验证 (E)

数据量25倍，时间约10倍 (184s vs 1800s)，Spark亚线性扩展。RMSE~0.852略优于ml-1m。API日志显示ml-25m健康检查75-92ms，推荐接口47-55ms。

五、综合分析

算法改进与创新点

- ALS+Content混合推荐**：在ALS候选池内通过 α 参数 (0.7) 加权融合Jaccard内容评分，排序指标提升5-6%。
- 冷启动自动切换**：预检测冷用户，推荐阶段自动从ALS切换到Jaccard内容推荐，覆盖率0%→100%。
- 6维全面评估**：在RMSE外增加排序指标、覆盖率和多样性评估。
- Left anti join过滤**：高效排除已评分电影。

方案对比分析

ALS基线Precision@10=0.31，调参确认默认最优。冷启动覆盖+10.8pp最显著。混合推荐排序+5-6%、多样性+9.4%，为最佳综合方案。

六、总结与展望

实现了ALS协同过滤、冷启动内容推荐、混合推荐三种策略。混合推荐排序+5-6%，冷启动覆盖+10.8pp，API响应<100ms。未来方向：Lambda架构实时推荐、增量模型更新、NCF深度学习、多模态特征融合、在线A/B测试。

附录：核心算法代码及说明

本章节展示 `train_als.py` 和 `app.py` 中关键函数的实现代码与设计说明。完整可运行代码请参见项目文件。

训练脚本 (train_als.py)

1. init_spark() - Spark初始化与跨平台适配

自动适配Windows/Linux。Windows下重定向临时目录、配置Hadoop兼容路径；Java 17+场景添加--add-opens模块开放选项。

```
def init_spark(driver_memory: str, shuffle_partitions: int = 16) -> SparkSession:
    builder = (SparkSession.builder
        .master("local[*,4]")
        .config("spark.driver.memory", driver_memory)
        .config("spark.sql.shuffle.partitions", str(shuffle_partitions)))

    if sys.platform == "win32":
        import tempfile
        win_tmp = tempfile.gettempdir()
        builder.config("spark.sql.warehouse.dir",
            "file:/// " + win_tmp.replace("\\", "/") + "/spark-warehouse")
        builder.config("spark.local.dir", os.path.join(win_tmp, "spark-tmp"))

    return builder.getOrCreate()
```

2. load_ratings() - 智能格式检测与数据加载

通过路径名自动检测数据集格式：ml-1m使用":"分隔符和ISO-8859-1编码，ml-25m使用","分隔符和UTF-8编码。Spark原生的CSV reader处理25M的数据头，RDD map +

split处理1M的非标准分隔符。

```
def detect_format(data_dir: str):
    if "25m" in data_dir.lower():
        return ",", "UTF-8"
    return ":", "ISO-8859-1"

def load_ratings(spark, data_dir):
    csv_path = os.path.join(data_dir, "ratings.csv")
    dat_path = os.path.join(data_dir, "ratings.dat")
    if os.path.exists(csv_path):
        return
    spark.read.option("header", True).csv(csv_path).select("userId", "movieId", "rating")
    elif os.path.exists(dat_path):
        return (spark.read.text(dat_path).rdd
                .map(lambda r: r[0].split(":"))
                .filter(lambda p: len(p) >= 3)
                .map(lambda p: (int(p[0]), int(p[1]), float(p[2])))
                .toDF(["userId", "movieId", "rating"]))
```

3. train_als() - ALS模型训练核心

coldStartStrategy="drop"表示丢弃测试集中未出现在训练集的用户/物品的预测；nonnegative=True约束因子矩阵非负，提升可解释性；seed=42固定随机种子保证可重复。

```
def train_als(train, args):
    als = ALS(
        maxIter=args.max_iter, regParam=args.reg_param, rank=args.rank,
        userCol="userId", itemCol="movieId", ratingCol="rating",
        coldStartStrategy="drop", seed=42,
        nonnegative=True, implicitPrefs=False)
    model = als.fit(train)
    return model
```

4. recommend_content_based() - Jaccard内容推荐

核心思路：通过Spark UDF定义Jaccard函数（计算两个类型字符串的交并比），用Cross Join计算所有(冷用户, 电影)对的类型相似度，用left anti join排除已评分电影，最后通过Window函数按分区排序取Top-N。

```
_jaccard = udf(
    lambda a, b: len(set(a.split("|")) & set(b.split("|")))
        / max(len(set(a.split("|"))) | set(b.split("|"))), 1)
    if a and b else 0.0, DoubleType())
```

```
user_profile = (ratings.join(movies, "movieId")
    .withColumn("genre", explode(split(col("genres"), "\\|"))))
    .groupBy("userId").agg(concat_ws("|",
collect_set("genre")).alias("user_genres"))))

scored = user_profile.crossJoin(movies.select("movieId",
col("genres").alias("movie_genres")))
scored = scored.withColumn("jaccard", _jaccard(col("user_genres"),
col("movie_genres")))
filtered = scored.join(ratings.select("userId", "movieId").distinct(),
["userId", "movieId"], "left_anti")

window = Window.partitionBy("userId").orderBy(desc("jaccard"))
result = filtered.withColumn("rn", row_number().over(window)).filter(col("rn") <=
top_n)
```

left anti join是关键性能优化：它只保留左表中未在右表出现的行，利用Spark Catalyst优化器的谓词下推，比逐条NOT IN过滤高效得多。

5. recommend_hybrid() - 混合推荐核心

混合推荐不改变ALS模型本身，而是在ALS生成的候选列表（top_nx5）内通过内容相似度做后处理重排序。ALS分数和内容分数各自按用户做min-max归一化，消除量纲差异，再通过alpha参数加权融合。


```

def recommend_hybrid(model, ratings, movies, top_n, alpha=0.7):
    fetch_n = top_n * 5
    # Step 1: ALS候选池（排除已评分）
    als_pool = (model.recommendForAllUsers(fetch_n)
                .select(explode("recommendations").alias("rec"))
                .select(col("rec.movieId").alias("movieId"),
                        col("rec.rating").alias("als_score")))
    .join(rated, ["userId", "movieId"], "left_anti"))
    # Step 2: 候选池内计算Jaccard内容分
    pool = als_pool.join(user_profile, "userId").join(movies, "movieId")
    pool = pool.withColumn("content_score", _jaccard(col("user_genres"),
                                                    col("genres")))
    # Step 3: 用户内min-max归一化
    stats = pool.groupBy("userId").agg(
        min("als_score").alias("a_min"), max("als_score").alias("a_max"),
        min("content_score").alias("c_min"), max("content_score").alias("c_max"))
    normed = pool.join(stats, "userId").withColumn("hybrid_score",
        alpha * (col("als_score") - col("a_min")) / (col("a_max") - col("a_min") +
        1e-9)
        + (1-alpha) * (col("content_score") - col("c_min")) / (col("c_max") -
        col("c_min") + 1e-9))
    # Step 4: 重排序取Top-N
    return (normed.withColumn("rn",
        row_number().over(Window.partitionBy("userId").orderBy(desc("hybrid_score"))))
        .filter(col("rn") <=
        top_n).select("userId", "movieId", col("hybrid_score").alias("predRating")))

```

$\alpha = 0.7$ 的物理含义：ALS评分占70%权重，内容评分占30%。冷用户（<5条评分）自动跳过此函数，直接调用recommend_content_based()走纯内容推荐。

6. evaluate_model() - 回归评估与排序评估

计算RMSE（评分预测精度）和Precision/Recall/NDCG@K（推荐排序质量）。相关电影定义为评分 ≥ 3.5 的电影。RankingMetrics来自Spark MLlib，提供分布式版本的排序指标计算。

```

def evaluate_model(model, test):
    predictions = model.transform(test)
    evaluator = RegressionEvaluator(metricName="rmse", labelCol="rating",
    predictionCol="prediction")
    rmse = evaluator.evaluate(predictions)
    logger.info(f"RMSE = {rmse:.4f}, 有效预测={predictions.count()}/{test.count()}")
    return rmse

def compute_ranking_metrics(model, test, k=10):
    test_users = test.select("userId").distinct()
    user_recs = model.recommendForUserSubset(test_users, k)
    pred = user_recs.select("userId",
    col("recommendations.movieId").alias("pred_items"))
    actual = test.filter(col("rating") >=

```

```

3.5).groupBy("userId").agg(collect_list("movieId").alias("actual_items"))
joined = pred.join(actual, "userId", "inner")
metrics = RankingMetrics(joined.select("pred_items", "actual_items").rdd
    .map(lambda r: (list(r.pred_items), list(r.actual_items))))
return metrics.precisionAt(k), metrics.recallAt(k), metrics.ndcgAt(k)

```

7. main() - 全流程调度

```

def main():
    args = parse_args()
    spark = init_spark(args.driver_memory, args.shuffle_partitions)
    ratings = load_ratings(spark, args.data_dir)
    movies = load_movies(spark, args.data_dir)
    analyze_cold_start(ratings) # 冷启动分析
    train, test = ratings.randomSplit([0.8, 0.2], seed=42)
    model = tune_hyperparams(train) if args.tune else train_als(train, args)
    evaluate_model(model, test) # 回归评估
    compute_ranking_metrics(model, test) # 排序评估
    if args.hybrid: # 混合推荐
        user_recs = recommend_hybrid(model, ratings, movies, args.top_n,
args.alpha)
    else: # ALS+冷启动fallback
        user_recs = generate_recommendations(model, ratings, args.top_n)
        cold_recs = recommend_content_based(ratings, movies, args.top_n)
        if cold_recs: user_recs = user_recs.join(cold_recs,
"userId", "left_anti").unionByName(cold_recs)
        compute_coverage_diversity(user_recs, movies, args.top_n) # 覆盖/多样性
        save_outputs(user_recs, movies, model, args.output_dir)

```

服务脚本 (app.py)

1. load_csv_dir() - CSV目录加载与分隔符自动检测

Spark输出的CSV分布在多个part-文件中。本函数自动合并所有part-文件。电影文件用|分隔（避免标题中的逗号导致解析错误），推荐文件用,分隔。

```

def load_csv_dir(dir_path: str) -> pd.DataFrame:
    frames, sample = [], None
    for fname in sorted(os.listdir(dir_path)):
        if fname.startswith("part-") and fname.endswith(".csv"):
            if sample is None:
                with open(os.path.join(dir_path, fname)) as f:
                    sample = "|" if "|" in f.readline() else ","
            frames.append(pd.read_csv(os.path.join(dir_path, fname), sep=sample))
    return pd.concat(frames, ignore_index=True)

```

2. init_data() - 双模式数据加载

内存模式（db_path=None）将全部推荐结果加载到全局dict，查询O(1)；SQLite模式适合25M大数据集，按userId使用B-tree索引查询，内存占用低且支持多进程共享。

3. recommend 路由 - 推荐接口

```
@app.route("/recommend/<int:user_id>")
def recommend(user_id):
    limit = max(1, min(request.args.get("limit", 10, type=int), 50))
    rec_list = _query_recs(user_id, limit)
    if not rec_list:
        return jsonify({"error": f"User {user_id} not found"}), 404
    for rec in rec_list:
        rec.update(get_movie_details(rec["title"], rec.get("genres", "")))
    return jsonify({"user_id": user_id, "recommendations": rec_list})
```

get_movie_details() 通过TMDB API获取电影海报、简介等信息。网络不可用时降级到Pollinations.ai生成AI海报，确保前端始终有视觉内容展示。

4. 日志中间件 - 结构化请求日志

通过Flask的before_request/after_request钩子记录每次请求的method、path、status、耗时和业务上下文（userId、推荐数等），日志文件10MB自动轮转。

路由汇总：

路由	功能	返回格式
GET /recommend/:id	个性化推荐（含海报）	JSON
GET /movie/:id	电影元信息	JSON
GET /health	服务器健康状态	JSON
GET /	复古电影院前端页	HTML

实验报告完